Knowledge Distillation And Deployment of a Lightweight Command-Polling Language Model on a Local Cloud

CS3081- Cloud Computing Mini Project

Samar Hidayat
23010104
Sec- A
9 Nov 2026.

1. Abstract :

I developed a lightweight ~~AI~~ natural-lang command parser using knowledge distillation and deployed it on a local kubernetes cloud. Used Phi 3.5 as a teacher model to generate structured training ~~model~~ labels & fine-tuned Python-70 M as a compact student model. Exposed inference through a Flask API, packaged the servic with docker and used K3d kubernetes for orchestration scaling, storage integration and recovery testing.

2. Architecture and implementation :

pipeline : Command dataset → Phi 3.5 teacher
→ JSON labels → Python-70M training
→ evaluation → Flask API → Docker Image
→ Kubernetes servic and Ingress.

3. Model Evaluation

Metric	Baseline	Student V1	Student V2
JSON validity	0%	100%	100%
Exact-frame accuracy	0%	0%	36%
Field accuracy	0%	46.5%	77%

4. Flask, Docker & Kubernetes

I exposed POST /parse, GET /health, and GET /ready through Flask and packaged the CPU-based model in a docker image (measured size 1.94 GB). Deployed it in K3d with CPU/memory limits, liveness/readiness probes, and working Ingress routing.

5. Autoscaling and fault recovery

I configured the Horizontal Pod Autoscaler for 1–2 replicas with a 50% CPU target. A controlled stress test raised the deployment from one to two replicas. I triggered an authenticated app crash & verified auto-recovery. Kubernetes restarted the container & returned it to 1/1 running with recorded restarts. I subsequently disabled fault injection.

6. Storage And Fetch Parallel Workers

I deployed Seaweed FS with persistent storage & uploaded datasets, teacher outputs, and student model-artifacts. Two indexed workers read the

headers, output dataset, produced upsert shards, and failed integrity checks. covering all 3000 records.

7. Networking and Automated Tests.

I designed four non-overlapping subnets: app 10.20.0.0/22 (1022 hosts), public 10.20.4.0/24 (254), data 10.20.5.0/25 (126) & management 10.20.5.128/27 (30). My NACL simulator included a deny rule for 198.51.100.0/24; all 6 packet checks and 3 Python unit tests passed.

8. performance & Cost assessment

I designed CPU based interface with bound resources & horizontal scaling. For capacity planning used a 95% successful-request target and P95 latency of 5 sec. these are design targets, not measured load-test results. Running locally avoids direct cloud dollar costs, while cloud deployment would incur compute & storage costs.

9. Result, Conclusion & Future Scope.

Final student Achieved 100% JSON validity, 77% field accuracy, 36% exact name accuracy, I also verified Ingest Accels, autoscaling, persistent object storage, Parallel Processing, network ~~protocols~~ controls, crash recovery. This demonstrates a functioning cloud-native command pallet further work include improving exact-name accuracy, benchmarking with locust, CI/CD automation, and loading model artifacts directly from object storage.

GitHub Repository:

→ https://github.com/F-h-d-hm-d/cloud-computing-mini-project